# Trees and Tries

**Topic 5 of 10**

**10 questions | skeleton and writing space only**

**Each question: the prompt, the function or class signature, and ruled lines to write your own solution.**

## 1. URL Request Handler Router

**MEDIUM | asked 2026-10-16 | string, parsing, data-structure, trie, dfs**

**Examples: Register /users/{userId}/photo/{photoId}. A lookup for /users/123/photo/456 matches that registered handler.**

```
class Router:

    def __init__(self):
```

```
    def add(self, pattern, handler=None, method
      ='GET'):
```

```
    def match(self, url, method='GET'):
```

## 2. Expression Tree Single-Leaf Mutation

**MEDIUM | asked 2026-09-27 | tree, dfs, simulation, state-machine**

**Evaluate a boolean expression tree, then answer the root value after each single-leaf flip.**

```
def flip_answers(root):
```

### 3. Compute File System Total Size

**MEDIUM | asked 2026-09-02 | tree, dfs**

**Examples: root = Dir( Dir(File(10), File(20)), File(5), Dir(File(30)), ) Total: 65**

```
def total_size(root, count_once=True):
```

### 4. Nested List Weight Sum (LC 339) with Follow-Ups

**MEDIUM | asked 2026-08-16 | tree, recursion, parsing**

**Examples: "[3, 8, [2, 14], [2, [91]]]" -> 3x1 + 8x1 + 2x2 + 14x2 + 2x2 + 91x3 = 320. Use [2, [91]] as a manual trace for bracket depth and the final-number flush.**

```
def weight_sum(nested):
```

## 5. Longest Consecutive Path in Binary Tree

**MEDIUM | asked 2026-07-02 | tree, dfs, recursion**

Given a binary tree, return the length of the longest parent-to-child path whose values increase consecutively by 1.

```
def longest_consecutive(root, want_path=False):
```

## 6. Trie Prefix Search / Autocomplete

**MEDIUM | asked 2026-07-02 | trie, tree, dfs**

LeetCode 208 + 1268. Build a trie from a dictionary and return all words sharing a query prefix. Standard Google onsite coding. Build a trie from a dictionary / array of words. Support searchPrefix(prefix) -> list[str]: return all words that start with the query prefix, lexicographically sorted unless ranking is specified.

```
class Trie:

    def __init__(self, k=3):

    def insert(self, word, freq=1):

    def find(self, prefix):

    def search_prefix(self, prefix):

    def suggest(self, prefix):

    def fuzzy(self, prefix):

    def search_below(self, node, text):
```

## 7. Greedy Longest-Match Dictionary Tokenizer

**MEDIUM | asked 2026-06-15 | string, trie, hashmap, greedy**

**Phone-screen coding round (R1). Input: a text string and a dictionary mapping key (string) -> id. Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one. Output the concatenation (or list) of ids and literal characters in order. Adapt the data structure so the rule swap is cheap.**

```
class Tokenizer:

    def __init__(self, table, priority=None):

    def better(self, new, old):

    def longest_at(self, text, i):

    def tokenize(self, text):
```

## 8. Employee Hierarchy Add / Delete

**MEDIUM | asked 2026-06-10 | tree, hashmap, object-design**

**Design an employee hierarchy class that supports adding employees under managers and deleting employees by id.**

```
class Org:

    def __init__(self, ceo):

    def add(self, emp, mgr):

    def senior(self, people):

    def delete(self, emp, policy='grandparent')
      :

    def reassign(self, emp, new_mgr):

    def move(self, emp, new_mgr):
```

## 9. Streaming Insert + Find K-th Largest

**MEDIUM | asked 2026-02-04 | data-structure, tree**

**Examples: insert(3) insert(3) insert(2) findLargest(0) -> 3 findLargest(1) -> 3 findLargest(2) -> 2**

```
class StreamKth:

    def __init__(self):
```

```
    def insert(self, x):
```

```
    def delete(self, x):
```

```
    def find_largest(self, k):
```

## 10. Huffman Tree Construction

**HARD | asked 2026-06-01 | tree, heap, greedy**

**Examples: {"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45} -> standard Huffman tree with f near the root.**

```
def huffman_codes(freq):
```